Week 6: Basic Docker Commands for Content Management

Aim:

To learn how to use simple Docker commands to manage applications inside containers.

Docker is the containerization platform that is used to package your application and all its dependencies together in the form of containers to make sure that your application works seamlessly in any environment which can be developed or tested or in production

Docker is a tool designed to make it easier **to create, deploy, and run applications by using containers.**

Containerization is a **software deployment process** that bundles an application's code with all the files and libraries it needs to run on any infrastructure.

Key Components of Docker

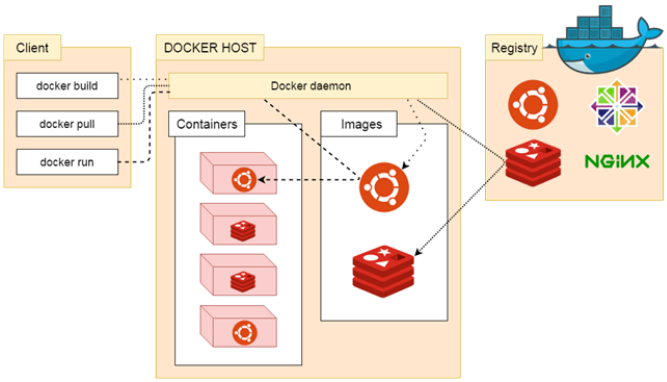
- **Docker Engine:** It is a core part of docker, that handles the creation and management of containers.
- **Container:**A container is a standard unit of software that packages up code and all its dependencies so the application runs quickly and reliably from one computing environment to another
- **Docker Image:** It is a read-only template that is used for creating containers, containing the application code and dependencies. It includes everything needed to run an application: ***code, runtime, system tools, system libraries and settings***.
- **Docker Hub:** It is a cloud-based repository that is used for finding and sharing the container images.
- **Docker file**: It is a script that containing instructions to build a docker image.
- **Docker Registry**: It is a storage distribution system for docker images, where you can store the images in both public and private modes

Docker daemon

Docker daemon runs on the host operating system. It is responsible for **running containers to manage docker services**. It offers various Docker objects such as **images, containers, networking, and storage**

Docker architecture

Docker follows Client-Server architecture, which includes the three main components that are **Docker Client, Docker Host, and Docker Registry.**



1. Docker Client

Docker client uses commands and REST APIs to communicate with the Docker Daemon (Server).

2. Docker Host

Docker Host is used to provide an environment to execute and run applications. It contains the docker daemon, images, containers, networks, and storage.

3. Docker Registry

Docker Registry manages and stores the Docker images.

There are two types of registries in the Docker -

Pubic Registry - Public Registry is also called as **Docker hub**.

Private Registry - It is used to share images within the enterprise.

Basic Docker Commands

1 docker --version

docker -v

This command is used to get the current version of the docker

```
PS C:\> docker -v
Docker version 20.10.17, build 100c701
PS C:\> docker --version
Docker version 20.10.17, build 100c701
```

2 **docker pull:** Pull an image or a repository from a registry

Syntax:docker pull [OPTIONS] NAME[: TAG|@DIGEST]

To download an image or set of images (i.e. A Repository)

Ex: **docker pull hello-world**

```
PS C:\> docker pull hello-world
Using default tag: latest
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest: sha256:d211f485f2dd1dee407a80973c8f129f00d54604d2c90732e8e320e5038a0348
Status: Downloaded newer image for hello-world:latest
docker.io/library/hello-world:latest
```

3 **docker run: This** command is used to create a container from an image

Syntax : docker run [OPTIONS] IMAGE [COMMAND] [ARG...]

Example : docker run hello-world

```
PS C:\> docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
c1ec31eb5944: Pull complete
Digest: sha256:d211f485f2dd1dee407a80973c8f129f00d54604d2c90732e8e320e5038a0348
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.
```

4. **docker ps**This command is used to list all the containers

Syntax : **docker ps [OPTIONS]**

Example :**docker ps (lists currently running containers)**

```
PS C:\> docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED   STATUS    PORTS     NAMES
```

Command : docker ps -a -a or -all : Lists all containers that are running & stopped

```
PS C:\> docker ps -a
CONTAINER ID   IMAGE         COMMAND    CREATED         STATUS                     PORTS     NAMES
e966ebcc530a   hello-world   "/hello"   4 minutes ago   Exited (0) 4 minutes ago             unruf
fled_spence
```

5.**docker restart**

This command is used to restart one or more containers.

Syntax: docker restart [OPTIONS] CONTAINER [CONTAINER...]

Example:docker restart my_container

6 docker kill

This command is used to kill one or more containers.

Syntax: docker kill [OPTIONS] CONTAINER [CONTAINER...]

Example:$ docker kill my_container

7Build an Image from a Dockerfile

docker build -t <image_name>

Ex: docker build -t

8 List local images

docker images

```
PS C:\> docker images
REPOSITORY              TAG       IMAGE ID       CREATED         SIZE
hello-world             latest    d2c94e258dcb   17 months ago   13.3kB
docker/getting-started  latest    3e4394f6b72f   22 months ago   47MB
```

9docker search To search for an image in Docker Hub.

docker search <image_name>

Ex: **docker search hello-world**

10Delete an Image

docker rmi <image_name>

Ex: **docker rmi hello-world**

Login into Docker

docker login -u <username>

Publish an image to Docker Hub

docker push <username>/<image_name>

Container commands

Create and run a container from an image, with a custom name:

docker run --name <container_name><image_name>

Run a container with and publish a container's port(s) to the host.

docker run -p <host_port>:<container_port><image_name>

Run a container in the background

docker run -d <image_name>

Start or stop an existing container:

docker start|stop <container_name> (or <container-id>)

Remove a stopped container:

docker rm <container_name>